UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

MARIA JAMILLI LEMOS DE MACEDO

PLATAFORMA DE PESQUISA AVANÇADA: FACILITANDO O ACESSO À INFORMAÇÃO A PARTIR DE FILTROS E MACHINE LEARNING

NATAL - RN
2024

MARIA JAMILLI LEMOS DE MACEDO

PLATAFORMA DE PESQUISA AVANÇADA: FACILITANDO O ACESSO À INFORMAÇÃO A PARTIR DE FILTROS E MACHINE LEARNING

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador: Prof. Dr. Luiz Affonso Guedes.

NATAL - RN

2024

Esta obra está licenciada com uma licença Creative Commons Atribuição 4.0 Internacional. Esta licença permite que outros distribuam, remixem, adaptem e desenvolvam seu trabalho, mesmo comercialmente, desde que creditem a você pela criação original. Link dessa licença: creativecommons.org/licenses/by/4.0/

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Macedo, Maria Jamilli Lemos de.
Plataforma de Pesquisa Avançada: facilitando o acesso à informação a partir de filtros e machine learning / Maria Jamilli Lemos de Macedo. - 2025.
49f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Departamento de Engenharia de Computação e Automação, Engenharia de Computação, Natal, 2025.
Orientação: Dr. Luiz Affonso Henderson Guedes de Oliveira.

1. Machine Learning - Monografia. 2. Desenvolvimento Web - Monografia. 3. Sistema de Recomendação - Monografia. 4. Busca Acadêmica - Monografia. I. Oliveira, Luiz Affonso Henderson Guedes de. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

MARIA JAMILLI LEMOS DE MACEDO

PLATAFORMA DE PESQUISA AVANÇADA: FACILITANDO O ACESSO À INFORMAÇÃO A PARTIR DE FILTROS E MACHINE LEARNING

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Aprovada em: 16 / 01 / 2025

BANCA EXAMINADORA

Prof. Dr. Luiz Affonso Guedes

Orientador(a)

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Eng. Felipe Lima de Medeiros

Membro interno

PPGEEC-UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Eng. Lemyson Oliveira Lemos

Membro interno

PPGEEC-UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Dedico este trabalho à minha mãe, que com tanto amor, força e dedicação, sempre esteve ao meu lado, acreditando nos meus sonhos e lutando para que eu pudesse alcançá-los.

AGRADECIMENTOS

A realização deste trabalho só foi possível graças ao apoio, à fé e à dedicação de muitas pessoas que, de diversas formas, contribuíram para que eu chegasse até aqui.

Em primeiro lugar, agradeço a Deus e a Nossa Senhora, pela força, proteção e bênçãos em minha vida. Sem a fé e a confiança em Seus planos, os desafios enfrentados ao longo desta jornada seriam muito mais difíceis.

Agradeço também à minha família, que sempre esteve ao meu lado, me apoiando em todas as etapas da minha vida. À minha mãe, Joelma, minha maior inspiração, que dedicou 100% de seus esforços à minha criação, muitas vezes abrindo mão de seus próprios desejos para que eu pudesse alcançar os meus. Sua força e amor incondicional são o que me motivam todos os dias. Aos meus avós, cuja sabedoria e carinho foram fundamentais para me formar como pessoa e me ensinaram valores que carrego comigo para sempre.

Quero também expressar minha gratidão aos meus amigos, que estiveram ao meu lado nos momentos de desafios e conquistas. A amizade e o incentivo de vocês foram essenciais para que eu mantivesse a motivação e a confiança em mim mesma.

Agradeço especialmente ao Professor Luiz Affonso, meu orientador, pela paciência, orientação e incentivo ao longo deste trabalho. Seu suporte foi fundamental para que este projeto se tornasse realidade.

Por fim, agradeço a todos que, direta ou indiretamente, contribuíram para a realização deste trabalho, com palavras de apoio, conselhos ou simplesmente acreditando no meu potencial.

A todos vocês, minha mais profunda gratidão.

"If one thing had been different
Would everything be different today?"
Taylor Swift

RESUMO

O presente trabalho apresenta o desenvolvimento do ProLeitura, um sistema de busca acadêmica inteligente voltado para estudantes e pesquisadores da Universidade Federal do Rio Grande do Norte (UFRN). A plataforma tem como objetivo otimizar o processo de busca por referências relevantes e confiáveis, utilizando aprendizado de máquina e integração com ferramentas modernas, como Firebase e Google Custom Search API. A metodologia adotada abrangeu técnicas de engenharia de software, design centrado no usuário (User-Centered Design) e aprendizado de máquina. O sistema oferece funcionalidades como personalização de resultados, salvamento de referências e predição de relevância, proporcionando uma experiência eficiente e adaptativa. Os resultados indicam melhorias significativas na eficiência das buscas e na qualidade das referências obtidas, destacando o impacto positivo da tecnologia no ambiente acadêmico. Por fim, o estudo discute as limitações encontradas e sugere aprimoramentos futuros para expandir as capacidades do sistema.

Palavras-chave: Busca acadêmica, aprendizado de máquina, sistemas de recomendação, personalização, ProLeitura.

ABSTRACT

This study presents the development of ProLeitura, an intelligent academic search system aimed at students and researchers from the Federal University of Rio Grande do Norte (UFRN). The platform seeks to optimize the process of finding relevant and reliable references by leveraging machine learning and modern tools such as Firebase and Google Custom Search API. The adopted methodology incorporated software engineering techniques, user-centered design (UCD), and machine learning. The system offers features like personalized search results, reference saving, and relevance prediction, providing an efficient and adaptive user experience. Results indicate significant improvements in search efficiency and reference quality, highlighting the positive impact of technology in academic environments. Finally, the study discusses identified limitations and proposes future enhancements to expand the system's capabilities.

Keywords: Academic search, machine learning, recommender systems, personalization, ProLeitura.

LISTA DE FIGURAS

Figura 1 – Problema de Sobrecarga Informacional: Diagrama ilustrando o fluxo de informações desde as fontes até os desafios enfrentados pelos usuários……..... 15

Figura 2 – Tecnologias Utilizadas no Sistema: Diagrama mostrando a integração das tecnologias utilizadas, incluindo React, Flask, Firestore, e Google Custom Search API…..... 16

Figura 3 – Pipeline de Treinamento do Modelo: Fluxograma das etapas do treinamento do modelo de aprendizado de máquina, desde a coleta de dados até a avaliação…… 20

Figura 4 – Diagrama Geral da Arquitetura: Representação em alto nível da arquitetura do sistema ProLeitura…… 21

Figura 5 – Tela de home: Design em alto nível da tela inicial do ProLeitura…… 23

Figura 6 – Tela de onboarding: Design em alto nível da tela responsável pela criação de usuário……. 24

Figura 7 – Tela de search: Design em alto nível da ferramenta de search e os resultados da pesquisa…….. 25

Figura 8 – Tela de resultados salvos: Design em alto nível da tela de resultados de pesquisa salvos pelo usuário……. 25

Figura 9 – Modelo do Decision Tree: Código do treinamento do modelo da Decision Tree…… 31

Figura 10 – Modelo do Random Forest: Código do treinamento do modelo da Random Forest…… 32

Figura 11 – Caminho da Random Forest 0: Caminho de tomada de decisão da árvore 0……. 35

Figura 12 – Caminho da Random Forest 1: Caminho de tomada de decisão da árvore 1……. 35

Figura 13 – Caminho da Decision Tree: Caminho de tomada de decisão da árvore...……. 36

Figura 14 – Matriz de Confusão Random Forest: Ilustração da análise gráfica do desempenho do sistema……..……. 37

Figura 15 – Matriz de Confusão Decision Tree: Ilustração da análise gráfica do desempenho do sistema……..……. 37

Figura 16 – Design do Sistema: Gráfico do feedback dos usuários em relação ao User Experience…… 39

Figura 17 – Interface Gráfica: Feedback dos usuários sobre as expectativas das telas……... 39

Figura 18 – Resultados das Pesquisas: Resposta dos usuários sobre os resultados apresentados de suas pesquisas……. 40

Figura 19 – Salvar Resultados: Análise dos usuários em relação a funcionalidade de salvar resultados……... 40

Figura 20 – Comparação do Tempo Médio de Busca: Comparação do tempo médio gasto em buscas com a plataforma ProLeitura……. 41

Figura 21 – Comparação do Tempo Médio de Busca: Comparação do tempo médio gasto em buscas sem a plataforma ProLeitura…….. 42

LISTA DE TABELAS

Tabela 1 – Endpoints RESTful Implementados: Descrição dos principais endpoints disponíveis no backend, suas funcionalidades e métodos HTTP……..... 23

Tabela 2 – Comparação entre Random Forest e Decision Tree: Comparação dos resultados obtidos entre os modelos Random Forest e Decision Tree…… 34

SUMÁRIO

1	**INTRODUÇÃO**	14
2	**MATERIAIS, FERRAMENTAS E MÉTODOS**	16
2.1	**Ferramentas e Tecnologias**	16
2.1.1	React	16
2.1.2	Flask	17
2.1.3	Firestore	17
2.1.4	Google Custom Search API	17
2.1.5	Scikit-learn	17
2.1.6	Joblib	18
2.1.7	Google Cloud Run	18
2.1.8	Firebase Hosting	18
2.2	**Coleta e Armazenamento de Dados**	18
2.3	**Treinamento do Modelo de Predição**	19
2.4	**Integração do Backend com o Frontend**	20
2.5	**Implementação do Sistema de Recomendação**	20
3	**DESENVOLVIMENTO**	21
3.1	**Arquitetura do Sistema**	21
3.1.1	Diagrama Geral da Arquitetura	21
3.1.2	Frontend	21
3.1.3	Backend	22
3.1.4	Banco de Dados	22
3.1.5	Infraestrutura	23
3.2	**Processos de Desenvolvimento**	23
3.2.1	Planejamento e Design de UX/UI	23
3.2.1	Implementação do Frontend	26
3.2.3	Configuração do Firebase	26
3.2.4	Configuração do Google Cloud	28
3.2.5	Integração de Machine Learning	28
4	**RESULTADOS**	30
4.1	Avaliação do Modelo de Aprendizado de Máquina	30
4.2	Desempenho do Sistema	31

4.3 Experiência do Usuário (UX) …….. 32
4.4 Impacto no Processo de Busca Acadêmica …….. 34
4.5 Limitações Encontradas ……... 36
5 **CONSIDERAÇÕES FINAIS**..... 37

1 INTRODUÇÃO

A internet, desde sua popularização, tem desempenhado um papel central na disseminação de informações e na promoção do acesso ao conhecimento. De acordo com Castells (1996), a era da informação caracteriza-se pela interconexão em larga escala, permitindo a circulação de dados em volumes antes inimagináveis. Contudo, essa abundância de informações, muitas vezes descrita como "sobrecarga informacional", pode dificultar a identificação de dados relevantes e confiáveis, especialmente no contexto acadêmico.

Para estudantes e pesquisadores, o processo de busca por referências bibliográficas adequadas é fundamental, pois afeta diretamente a qualidade e a profundidade de seus trabalhos acadêmicos (Rowley e Hartley, 2017). No entanto, essa tarefa pode se tornar um desafio devido à grande quantidade de informações disponíveis e à dificuldade de identificar fontes confiáveis e relevantes. A sobrecarga informacional, como discutido por Sigolo e Casarin (2024), pode levar à sobrecarga cognitiva, dificultando a filtragem e a organização dos materiais encontrados em bases de dados extensas.

Além disso, a ausência de ferramentas que personalizem e refinem os resultados de busca agrava esse problema, aumentando o tempo necessário para localizar referências pertinentes e reduzindo a eficiência da pesquisa acadêmica. Estudos recentes indicam que, mesmo com o avanço das tecnologias de indexação e recuperação de informações, os sistemas atuais nem sempre atendem às necessidades específicas dos usuários, resultando em frustração e perda de produtividade (Tenopir et al., 2020).

Dessa forma, torna-se essencial o desenvolvimento de soluções tecnológicas que não apenas otimizem a busca, mas também auxiliem na curadoria e recomendação de conteúdos acadêmicos de forma personalizada. A falta dessas soluções impacta diretamente a qualidade dos trabalhos desenvolvidos e pode comprometer o aprofundamento das pesquisas realizadas.

Neste cenário, o presente trabalho propõe o desenvolvimento de um sistema inteligente de busca acadêmica, denominado **ProLeitura**, que visa otimizar o acesso a informações relevantes e confiáveis para os estudantes e pesquisadores da Universidade Federal do Rio Grande do Norte (UFRN). A plataforma utiliza tecnologias de aprendizado de máquina (machine learning) e integração com sistemas de busca acadêmicos para oferecer uma experiência personalizada e eficiente. Sistemas de recomendação, como apontam Ricci et

al. (2015), têm mostrado grande potencial em diversos domínios ao analisar o comportamento do usuário e sugerir conteúdos alinhados às suas preferências.

O problema central deste trabalho é: *como promover um acesso mais eficiente e direcionado a informações acadêmicas relevantes, considerando as necessidades individuais dos estudantes e pesquisadores?* Este projeto busca responder a essa questão desenvolvendo um sistema que não apenas filtra conteúdos, mas também aprende com as interações do usuário, oferecendo resultados cada vez mais precisos ao longo do tempo.

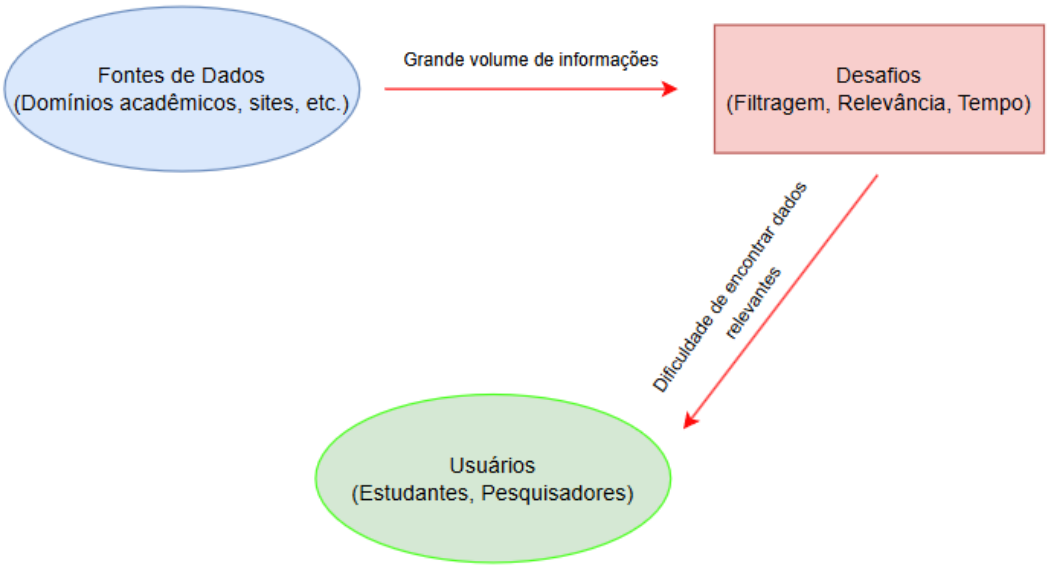

Figura 1 - Problema de Sobrecarga Informacional: Diagrama ilustrando o fluxo de informações desde as fontes até os desafios enfrentados pelos usuários."

A relevância deste estudo está fundamentada no impacto que soluções tecnológicas podem ter no aprimoramento das práticas acadêmicas. Segundo Krikelas (1983), a eficiência na recuperação de informações é um fator determinante para o sucesso das atividades de pesquisa. Assim, o desenvolvimento do **ProLeitura** busca contribuir para a superação desse desafio ao unir tecnologias modernas e necessidades específicas do público acadêmico. Por meio da análise crítica dos resultados obtidos e das contribuições da tecnologia na educação, este trabalho almeja gerar avanços tanto no acesso à informação quanto na qualidade do ensino e da pesquisa.

Por fim, este trabalho está estruturado em cinco seções principais. A Introdução apresenta o problema, os objetivos e a relevância da pesquisa no contexto acadêmico e tecnológico. Em Materiais e Métodos, são descritas as ferramentas utilizadas, como Firebase e

Google Custom Search API, bem como os métodos e abordagens teóricas que embasaram o desenvolvimento da solução. A seção de Desenvolvimento detalha a implementação do sistema ProLeitura, incluindo sua arquitetura, funcionalidades principais e integração de técnicas de aprendizado de máquina. Em Resultados e Discussões, são analisados os dados obtidos com a utilização do sistema, avaliando sua eficácia, limitações e o impacto percebido pelos usuários. Por fim, a Conclusão resume os objetivos alcançados, discute as principais limitações e apresenta sugestões para futuros aprimoramentos.

2 MATERIAIS, FERRAMENTAS E MÉTODOS

Nesta seção, são descritos os materiais, ferramentas e métodos utilizados no desenvolvimento do sistema de busca e recomendação para estudantes da UFRN. A abordagem metodológica adotada fundamenta-se em conceitos de aprendizado de máquina, recuperação de informações e engenharia de software, assegurando uma implementação eficiente e baseada em boas práticas.

2.1 Ferramentas e Tecnologias

Nesta seção, são apresentadas as ferramentas e tecnologias empregadas no desenvolvimento do sistema, com uma explicação detalhada sobre cada uma delas e suas aplicações no projeto.

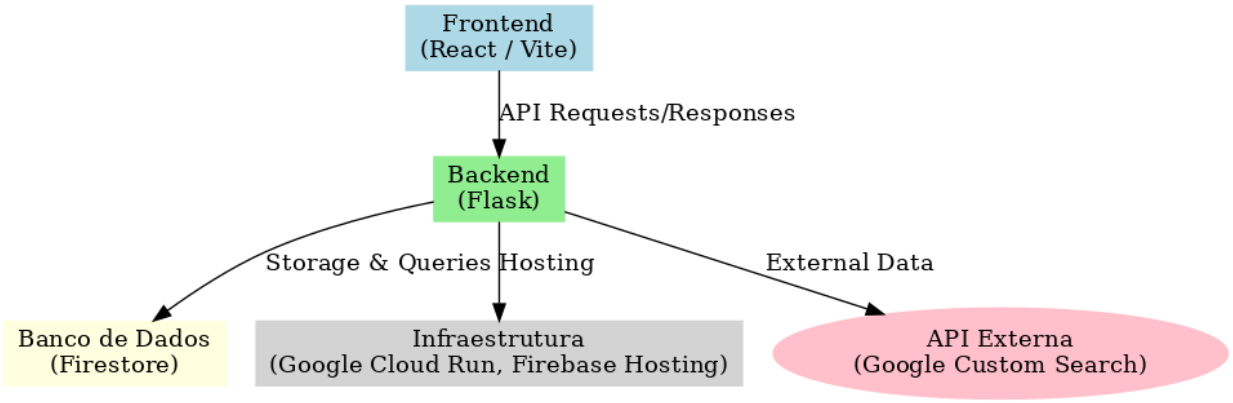

Figura 2 - Tecnologias Utilizadas no Sistema: Diagrama mostrando a integração das tecnologias utilizadas, incluindo React, Flask, Firestore, e Google Custom Search API.

2.1.1 React

O React é uma biblioteca JavaScript amplamente utilizada para a criação de interfaces de usuário. Desenvolvida pelo Facebook, ela permite a construção de componentes reutilizáveis, que facilitam a manutenção e expansão do sistema. Neste projeto, o React foi usado para criar o frontend do sistema, proporcionando uma experiência de usuário responsiva e eficiente. Além disso, sua capacidade de gerenciamento de estado e integração com outras bibliotecas foi fundamental para a implementação de funcionalidades complexas, como a exibição de resultados de busca e o gerenciamento de interações do usuário (ROBINSON, 2021).

2.1.2 Flask

O Flask é um microframework Python leve e flexível, amplamente utilizado para a criação de APIs RESTful. Ele foi escolhido para o backend devido à sua simplicidade e eficiência na implementação de serviços que conectam o frontend ao modelo de aprendizado de máquina. O Flask permitiu a criação de endpoints para o processamento de dados do usuário, a predição de relevância dos resultados e a integração com o Google Custom Search API (GRINBERG, 2018).

2.1.3 Firestore

O Firestore, parte do Firebase, é uma solução de banco de dados NoSQL gerenciada e altamente escalável. Neste projeto, ele foi utilizado para armazenar dados relacionados aos usuários, incluindo informações básicas, interações e resultados salvos. A escolha do Firestore deveu-se à sua capacidade de sincronização em tempo real e à facilidade de integração com serviços da Google Cloud Platform, permitindo o desenvolvimento de uma aplicação moderna e responsiva (GOOGLE, 2023).

2.1.4 Google Custom Search API

O Google Custom Search API foi utilizado para realizar buscas personalizadas na web, permitindo filtrar resultados com base em domínios específicos, como o site da UFRN, e palavras-chave relacionadas ao tema de interesse do usuário. Essa API ofereceu a flexibilidade necessária para fornecer resultados relevantes de maneira eficiente e escalável (GOOGLE, 2023).

2.1.5 Scikit-learn

O Scikit-learn é uma biblioteca Python renomada por suas ferramentas de aprendizado de máquina. Neste projeto, ela foi usada para o treinamento do modelo preditivo, utilizando técnicas como o Random Forest Classifier para avaliar a relevância dos resultados de busca. A biblioteca também foi empregada para pré-processamento de dados e avaliação de métricas, garantindo precisão e confiabilidade nos resultados (PEDREGOSA et al., 2011).

2.1.6 Joblib

A biblioteca Joblib foi empregada para serializar o modelo treinado, permitindo sua reutilização no ambiente de produção. Essa funcionalidade foi essencial para integrar o modelo preditivo ao sistema em tempo real, sem a necessidade de treinamentos repetitivos (GOOGLE, 2023).

2.1.7 Google Cloud Run

O Google Cloud Run é um serviço gerenciado que permite a execução de contêineres em uma infraestrutura escalável. Ele foi utilizado para hospedar o backend do sistema, garantindo alta disponibilidade e desempenho, mesmo sob cargas variáveis de requisições. Sua integração com outras ferramentas da Google Cloud Platform simplificou o gerenciamento do ambiente de produção (GOOGLE, 2023).

2.1.8 Firebase Hosting

O Firebase Hosting foi escolhido para hospedar o frontend do sistema. Ele oferece baixa latência, suporte a HTTPS e integração com o Firestore, tornando-se uma solução eficiente para disponibilizar o sistema aos usuários finais (GOOGLE, 2023).

2.1.9 Integração das Ferramentas

A integração entre essas ferramentas foi projetada para garantir uma solução robusta, escalável e fácil de manter. Cada tecnologia desempenhou um papel específico no ciclo de desenvolvimento, desde a coleta de dados até a apresentação de resultados relevantes ao usuário.

2.2 Coleta e Armazenamento de Dados

Os dados utilizados neste projeto foram coletados de forma ética e transparente, respeitando os princípios de privacidade dos usuários. As seguintes coleções foram configuradas no Firestore:

- **Users**: Contém informações básicas sobre o perfil dos usuários, como área de estudo, nível acadêmico e preferências.
- **Interactions**: Armazena dados sobre buscas realizadas e cliques em resultados relevantes. Esses dados são usados para treinar o modelo de aprendizado de máquina.
- **SavedResults**: Armazena os resultados que os usuários marcaram como relevantes, ajudando a personalizar futuras recomendações.

Todos os dados foram anonimizados e utilizados exclusivamente para fins de pesquisa e melhoria do sistema, conforme recomendado por Witten e Frank (2016).

2.3 Modelos de Predição e Treinamento

O **Decision Tree** é um modelo de aprendizado supervisionado amplamente utilizado em problemas de classificação e regressão. Ele organiza os dados em uma estrutura hierárquica semelhante a uma árvore, onde cada nó interno representa uma condição ou regra aplicada a uma variável, e cada folha representa uma classificação ou decisão. Sua principal vantagem é a interpretabilidade, já que é possível visualizar e compreender facilmente as decisões tomadas em cada etapa (MITCHELL, 1997).

Por outro lado, o **Random Forest** expande a abordagem do Decision Tree utilizando o conceito de ensemble learning. Este modelo constrói uma "floresta" de múltiplas árvores de decisão, cada uma treinada com amostras aleatórias do conjunto de dados (técnica de Bagging) e subconjuntos das variáveis explicativas. O resultado final é obtido pela média (para regressão) ou votação da maioria (para classificação) das previsões individuais de cada árvore. Essa técnica melhora a precisão, reduz a variância e o risco de overfitting, tornando o Random Forest altamente robusto e eficiente para problemas complexos e dados de alta dimensionalidade (BREIMAN, 2001).

O treinamento do modelo de aprendizado de máquina foi realizado com o objetivo de prever a relevância de resultados de busca para cada usuário. O processo seguiu as etapas descritas a seguir:

- **Pré-processamento dos Dados**:
  - Os dados coletados foram normalizados e transformados para adequação ao modelo preditivo. Técnicas de limpeza, como remoção de valores ausentes e transformação de texto em números, foram aplicadas conforme orientado por Han et al. (2022).
- **Divisão do Dataset**:
  - O conjunto de dados foi dividido em 80% para treinamento e 20% para teste, garantindo uma avaliação adequada do desempenho do modelo (HASTIE et al., 2009).
- **Modelos Utilizados**:
  - **Decision Tree**: Limitado a uma profundidade máxima de 5, equilibrando simplicidade e desempenho.
  - **Random Forest**: Composto por 100 árvores, maximizando a robustez por meio da agregação de múltiplas previsões.
- **Avaliação**:
  - Métricas como acurácia, precisão e recall foram utilizadas para avaliar a performance do modelo.

As implementações foram realizadas com o auxílio da biblioteca Scikit-learn, amplamente utilizada para tarefas de aprendizado de máquina. Os modelos foram salvos utilizando Joblib, garantindo sua reutilização e fácil integração no sistema final.

A Figura 3 ilustra o pipeline de treinamento do modelo de aprendizado de máquina utilizado no sistema ProLeitura. O fluxo detalha as etapas desde a coleta dos dados até a avaliação do modelo, garantindo a geração de recomendações personalizadas para os usuários.

O processo ocorre da seguinte forma:

1. **Coleta de Dados**: A primeira etapa envolve a aquisição de informações a partir de fontes externas, como o Google Custom Search e o Firestore. Esses dados consistem em referências bibliográficas, metadados e interações dos usuários no sistema, servindo como insumo para as próximas etapas.
2. **Pré-processamento**: Após a coleta, os dados passam por um processo de limpeza e transformação. Essa etapa inclui a remoção de inconsistências, normalização dos textos e organização dos dados em um formato adequado para análise. O objetivo é

garantir que as informações estejam estruturadas corretamente para o treinamento do modelo.

3. **Treinamento do Modelo (Random Forest)**: Com os dados devidamente preparados, inicia-se o treinamento do modelo Random Forest. Esse algoritmo de aprendizado supervisionado analisa padrões nos dados de entrada e constrói múltiplas árvores de decisão para melhorar a precisão das previsões. O treinamento do modelo ocorre automaticamente **duas vezes ao dia, à meia-noite e ao meio-dia**, garantindo que as recomendações sejam constantemente atualizadas. Esse processo é gerenciado pelo **Google Cloud Run**, que executa as rotinas de treinamento de maneira escalável e eficiente.
4. **Avaliação do Modelo**: Para garantir a eficácia do sistema, o modelo treinado é submetido a uma fase de avaliação. Nessa etapa, métricas de desempenho são utilizadas para medir a acurácia e a capacidade do modelo de gerar recomendações coerentes e alinhadas às necessidades dos usuários. Caso necessário, ajustes são realizados para otimizar os resultados.

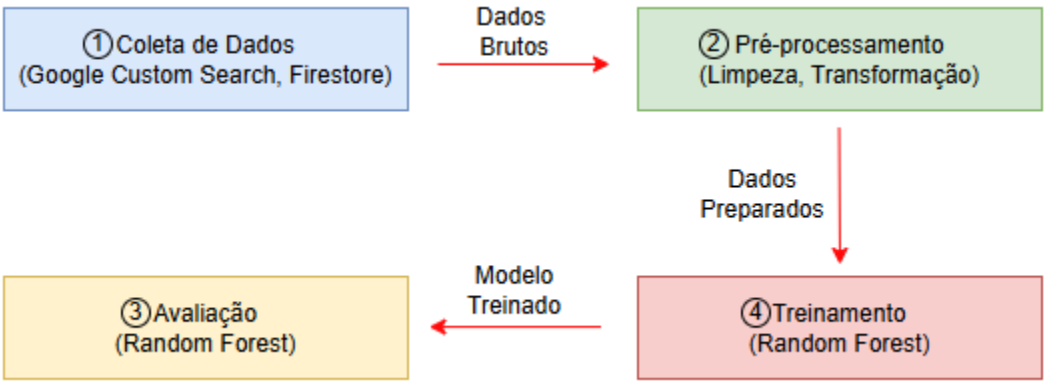

Figura 3 - Pipeline de Treinamento para o Modelo Random Forest: Fluxograma das etapas do treinamento do modelo de aprendizado de máquina, desde a coleta de dados até a avaliação.

2.4 Integração do Backend com o Frontend

A integração entre o frontend e o backend foi realizada por meio de uma API RESTful, que disponibiliza os seguintes serviços:

- Predição de relevância para personalização de resultados.

- Registro das interações dos usuários com o sistema.
- Busca em fontes externas por meio da integração com a Google Custom Search API.

Essa arquitetura foi projetada para ser escalável e garantir uma experiência de usuário otimizada (FIELDING, 2000).

2.5 Implementação do Sistema de Recomendação

O sistema de recomendação utiliza aprendizado de máquina combinado com regras baseadas em preferências dos usuários para personalizar os resultados. O algoritmo adapta os resultados apresentados com base nos seguintes fatores:

- Preferências declaradas pelo usuário, como área de estudo e nível acadêmico.
- Padrões de interação, incluindo buscas realizadas e resultados clicados.
- Dados históricos armazenados em coleções no Firestore.

A implementação do sistema foi orientada pelas melhores práticas de design de sistemas de recomendação, conforme descrito por Ricci et al. (2022).

3 DESENVOLVIMENTO

O desenvolvimento deste sistema foi conduzido com base em boas práticas de engenharia de software, utilizando ferramentas modernas para garantir escalabilidade, desempenho e facilidade de manutenção. Este capítulo descreve detalhadamente o processo de desenvolvimento, as decisões tomadas e os resultados alcançados.

3.1 Arquitetura do Sistema

A arquitetura foi projetada para ser modular, escalável e eficiente, dividida em três camadas principais:

3.1.1 Diagrama Geral da Arquitetura

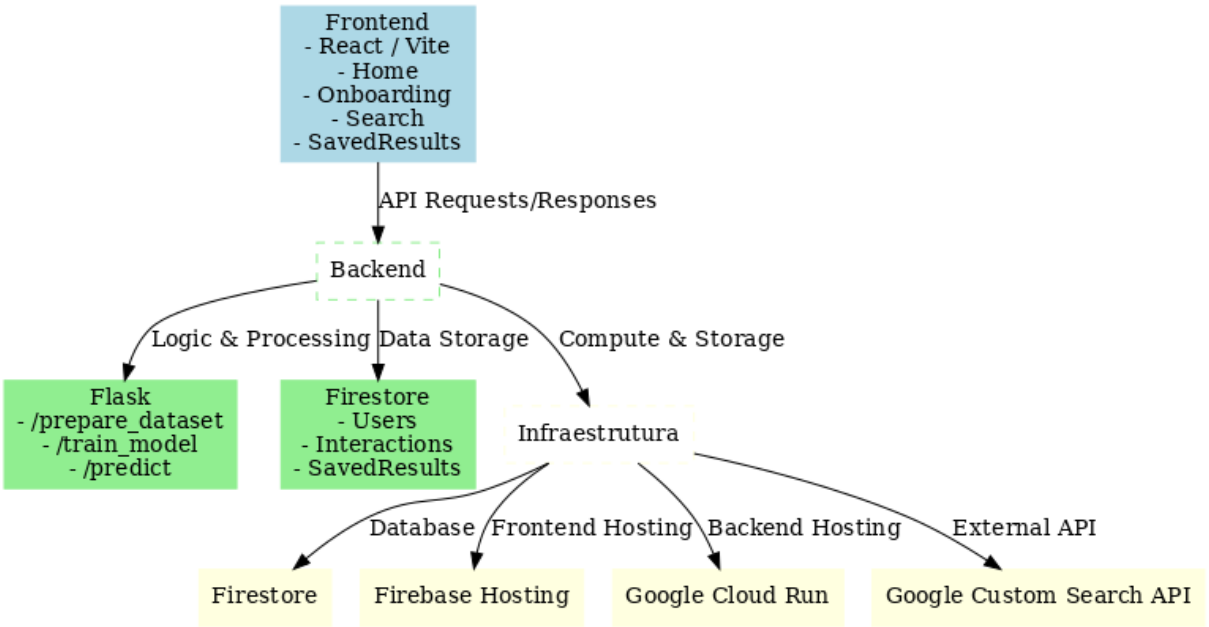

Figura 4 - Diagrama Geral da Arquitetura: Representação em alto nível da arquitetura do sistema ProLeitura.

3.1.2 Frontend

O frontend, construído com React, foi projetado seguindo princípios de componentização e modularidade. Os componentes foram divididos em:

- **Componentes Básicos**: Botões, campos de texto, modais, e toasts.
- **Componentes de Página**: Telas completas, como Home, Onboarding, Search e SavedResults.
- **Estado Global**: Utilização de ferramentas como Context API e Hooks para gerenciar estados globais, como autenticação do usuário e resultados de busca.

Para estilização, o Tailwind CSS foi utilizado devido à sua eficiência e flexibilidade. A abordagem mobile-first garantiu uma experiência responsiva para dispositivos de diferentes tamanhos.

3.1.2 Backend

O backend foi implementado com Flask, organizado em endpoints RESTful:

- **/prepare_dataset**: Processa os dados de interações e preferências para gerar um dataset.
- **/train_model**: Treina o modelo preditivo, salvando-o para uso futuro.
- **/predict**: Realiza a predição da relevância de resultados para buscas personalizadas.

O backend também incluiu integração com o modelo de aprendizado de máquina treinado utilizando Scikit-learn e serializado com Joblib.

O Firestore, banco de dados NoSQL do Firebase, foi utilizado para armazenar:

- **Usuários**: Dados de perfil, preferências e histórico.
- **Interações**: Registros de cliques em resultados de busca.
- **Resultados Salvos**: Informações sobre os conteúdos marcados como relevantes pelos usuários.

Elemento	Nome	Atributos	Descrição
Endpoint	/prepare_dataset	user_id (String)	**Método:** POST **Retorno:** JSON com o dataset preparado para treinamento do modelo preditivo.
Endpoint	/train_model	user_id (String)	**Método:** POST **Retorno:** Mensagem de sucesso ou erro, incluindo a acurácia do modelo treinado.
Endpoint	/predict	user_id (String), query (String)	**Método:** POST **Retorno:** Predição de relevância para a consulta informada.
Coleção	users	user_id (String), email (String), preferences (Array)	**Métodos:** GET, POST e PATCH **Finalidade:** Armazena informações básicas e preferências dos usuários.
Coleção	interactions	interaction_id (String),	**Métodos:** GET, POST e DELETE

		user_id (String), query (String), clickedResult (Object)	**Finalidade:** Registra as interações dos usuários com os resultados de busca, incluindo cliques e consultas realizadas.
Coleção	savedResults	result_id (String), user_id, title (String), link (String)	**Métodos:** GET, POST e DELETE **Finalidade:** Armazena resultados salvos pelos usuários para fácil acesso posterior.

Tabela 1 - Endpoints RESTful Implementados: Descrição dos principais endpoints disponíveis no backend, suas funcionalidades e métodos HTTP.

3.1.4 Infraestrutura

- **Google Cloud Run** foi usado para hospedar o backend, proporcionando escalabilidade automática e integração com o Firestore.
- **Firebase Hosting** garantiu alta disponibilidade e baixa latência para o frontend

3.2 Processo de Desenvolvimento

3.2.1 Planejamento e Design de UX/UI

O processo de design iniciou-se com a criação de wireframes e protótipos no Figma, garantindo que as telas fossem intuitivas e funcionais. Algumas decisões de design incluem:

- **Home**:
  - Destaque para o botão de login com autenticação via Google.
  - Explicação clara da proposta do sistema.

Figura 5 - Tela de home: Design em alto nível da tela inicial do ProLeitura.

- **Onboarding**:
  - Coleta de informações do usuário (curso, área de estudo, nível de formação, interesses e preferências).
  - Interface em formato de formulário dinâmico e intuitivo.

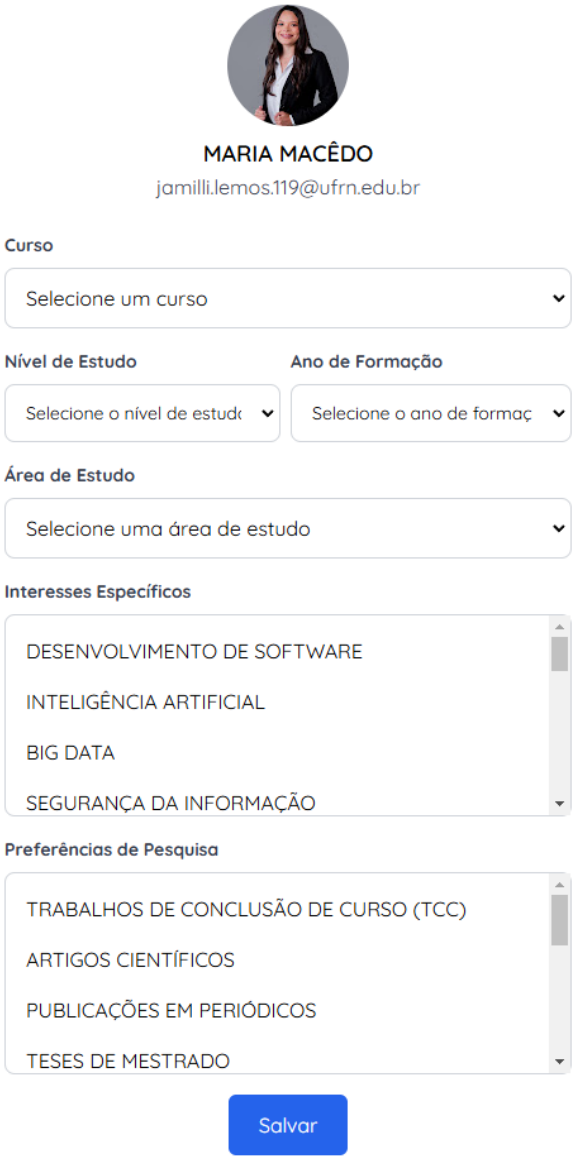

Figura 6 - Tela de onboarding: Design em alto nível da tela responsável pela criação de usuário.

- **Search**:
  - Campo de busca no topo, com personalização automática baseada nos dados do usuário.
  - Lista de resultados com botões para salvar ou visualizar detalhes.

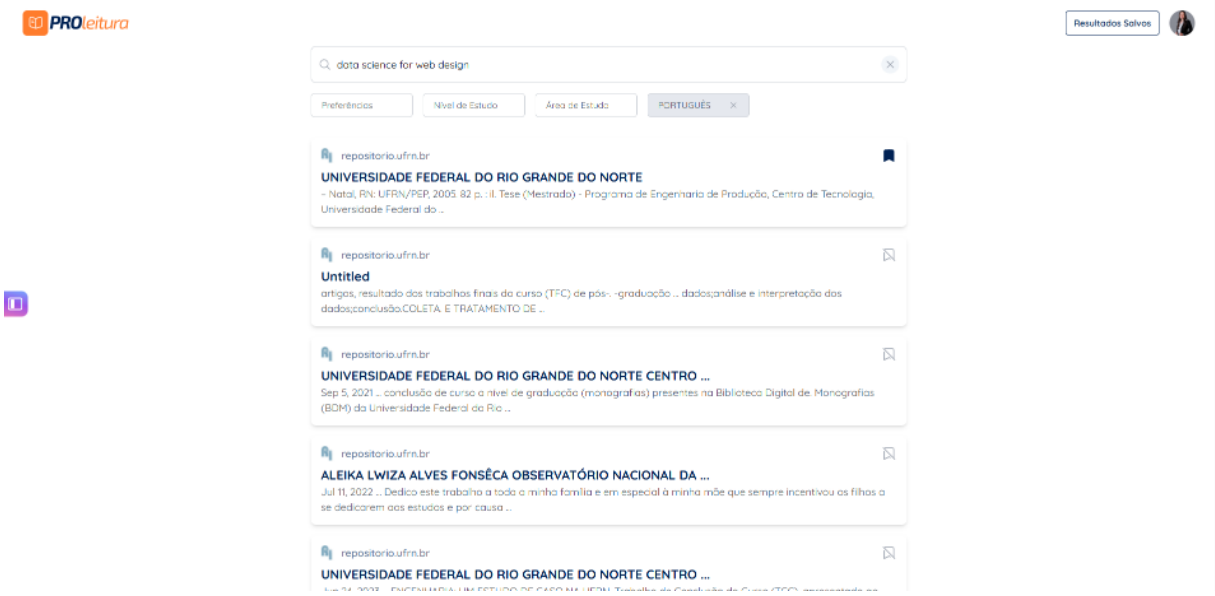

Figura 7 - Tela de search: Design em alto nível do search e os resultados da pesquisa.

- **SavedResults**:
  - Exibição dos resultados salvos, permitindo gerenciamento direto.

Figura 8 - Tela de resultados salvos: Design em alto nível da tela de resultados de pesquisa salvos pelo usuário.

Técnicas de User-Centered Design (UCD) foram aplicadas para garantir que o sistema atendesse às necessidades dos usuários, baseando-se em entrevistas com estudantes da UFRN para validar as funcionalidades.

3.2.2 Implementação do Frontend

A implementação seguiu práticas de Desenvolvimento Ágil, com entregas iterativas e testes constantes. Cada tela foi implementada com as seguintes funcionalidades:

- **Home**:
  - Integração com Firebase Authentication para login seguro.
  - Validação de domínios de e-mail autorizados.
- **Onboarding**:
  - Formulários dinâmicos que armazenam as preferências do usuário no Firestore.
  - Sistema de validação para garantir a consistência dos dados inseridos.
- **Search**:
  - Integração com a Google Custom Search API para buscar resultados relevantes.
  - Uso do modelo de predição para classificar a relevância dos resultados.
- **SavedResults**:
  - Funcionalidade para visualizar e remover itens salvos.
  - Atualização em tempo real, sincronizada com o Firestore.

3.2.3 Configuração do Firebase

O Firebase foi escolhido como uma solução integrada para a autenticação, armazenamento de dados e hospedagem do frontend devido à sua robustez, facilidade de uso e integração direta com outros serviços do Google Cloud Platform. A seguir, detalha-se o processo técnico de configuração e uso de cada módulo do Firebase.

1. **Firebase Authentication**

A autenticação foi implementada utilizando o serviço de Firebase Authentication com suporte para login via Google. Este método foi escolhido por garantir segurança, simplificar a experiência do usuário e permitir a restrição por domínio (apenas emails terminados em @ufrn.edu.br ou @ufrn.br são aceitos).

- **Fluxo de implementação técnica:**
  - A integração com o frontend foi realizada através da biblioteca firebase/auth, utilizando o método signInWithPopup, que retorna informações do usuário autenticado, como uid, email e token.

- Para validação de domínio, foi configurado no backend um filtro que bloqueia emails não permitidos, e o sistema força o logout caso a validação falhe.
- O token JWT gerado pelo Firebase Authentication é armazenado localmente e utilizado para proteger as chamadas à API do backend.

2. **Firestore (Banco de Dados)**

O Firestore foi utilizado para armazenar e sincronizar os dados em tempo real, estruturados em três coleções principais:

- **users:** Armazena informações básicas do usuário, incluindo preferências e nível de estudo.
- **interactions:** Registra as interações do usuário, como buscas realizadas e cliques em resultados.
- **savedResults:** Armazena os resultados salvos, permitindo que sejam acessados posteriormente.

A escolha do Firestore se deu por sua escalabilidade horizontal e capacidade de fornecer sincronização em tempo real, essencial para aplicações modernas. Além disso, as regras de segurança do Firestore garantem que apenas o próprio usuário possa acessar seus dados, adicionando uma camada extra de privacidade.

- **Exemplo técnico de estruturação:**
  - Cada documento em interactions contém campos como query, clickedResult (título, link, site), timestamp e userId.
  - As operações no banco foram otimizadas para minimizar custos, utilizando consultas indexadas e operações de batch para gravar múltiplos documentos.

3. **Firebase Hosting**

O Firebase Hosting foi configurado para hospedar o frontend React. Optou-se pelo Firebase Hosting devido à sua baixa latência e suporte para HTTPS por padrão, garantindo segurança no tráfego de dados entre cliente e servidor.

O processo de build do React foi integrado ao pipeline de deploy, gerando uma aplicação otimizada com caching configurado para recursos estáticos.

3.2.4 Configuração do Google Cloud

O Google Cloud foi responsável por hospedar o backend e integrar os serviços de machine learning e Firestore. A escolha pelo Cloud Run foi baseada na necessidade de um ambiente escalável e gerenciado, que reduz a complexidade operacional.

1. Cloud Run

O backend foi empacotado em um container Docker e hospedado no Google Cloud Run.

- **Vantagens técnicas:**
  - Escalabilidade automática com base na demanda.
  - Integração nativa com o Firestore através de contas de serviço.
  - Hospedagem segura com suporte a autenticação via JWT, permitindo que o backend restrinja acessos não autorizados.

2. Configuração de Contêiner

A imagem Docker do backend foi criada com o objetivo de ser leve e eficiente, utilizando uma base python:3.9-slim. O contêiner inclui o modelo de machine learning serializado (random_forest_model.pkl), permitindo que o serviço seja iniciado rapidamente e reduza a necessidade de treinamento frequente.

3. Integração com o Firestore

O backend foi configurado para acessar o Firestore utilizando a conta de serviço do Google Cloud. Foram implementadas restrições para garantir que apenas endpoints autenticados possam realizar leituras ou escritas no banco de dados.

Além disso, as permissões do IAM foram configuradas para conceder apenas os papéis necessários ao Cloud Run, seguindo o princípio do menor privilégio.

3.2.5 Integração de Machine Learning

O modelo de aprendizado de máquina é um dos diferenciais do sistema, permitindo classificar a relevância dos resultados de busca com base nas preferências e interações dos usuários.

1. Treinamento do Modelo

Para implementar a classificação de relevância das pesquisas, dois modelos de Machine Learning foram utilizados: **Decision Tree** e **Random Forest**. Ambos os modelos foram treinados com um dataset retirado das informações dos usuários e suas interações passadas armazenadas no Firestore a fim de classificar suas buscas em "Relevante" ou "Não Relevante".

- **Dados de entrada:** Queries realizadas, resultados clicados e tempo de interação.
- **Pré-processamento:** Normalização e limpeza dos dados para remover outliers e inconsistências.
- **Modelo:** Random Forest foi selecionado por sua robustez em cenários com dados heterogêneos, como texto e categorias. A fim de comparação, foi realizado testes com o Decision Tree.
- **Métricas:** A acurácia do modelo foi medida utilizando validação cruzada, atingindo cerca de 80% no conjunto de testes.

Treinamento do Decision Tree

Na Figura 9, é apresentado o código desenvolvido para o treinamento do modelo utilizando a técnica Decision Tree. Este modelo segue uma abordagem de aprendizado supervisionado, onde os dados são divididos em conjuntos de treinamento e teste. O código inclui:

- **Divisão dos dados**: Separação dos atributos preditores e da variável-alvo.
- **Criação do modelo**: Configuração da profundidade máxima da árvore e inicialização com um estado aleatório para reprodutibilidade.
- **Treinamento e avaliação**: Ajuste do modelo aos dados de treinamento e cálculo da acurácia com base nos dados de teste.

Essa implementação permite construir árvores de decisão eficientes para analisar padrões nos dados, servindo como um passo preliminar para métodos mais avançados, como o Random Forest.

```python
def train_and_plot_tree(df):
    X = df.drop(columns=["label"])
    y = df["label"]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    dt_model = DecisionTreeClassifier(
        max_depth=5,
        random_state=42
    )
    dt_model.fit(X_train, y_train)

    y_pred = dt_model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Acurácia da árvore de decisão: {accuracy}")

    print("Relatório de classificação:")
    print(classification_report(y_test, y_pred, target_names=["Não Relevante", "Relevante"]))

    print("Matriz de confusão:")
    print(confusion_matrix(y_test, y_pred))

    feature_names = X.columns.tolist()
    plt.figure(figsize=(20, 10))
    plot_tree(
        dt_model,
        feature_names=feature_names,
        class_names=["Não Relevante", "Relevante"],
        filled=True,
        rounded=True,
        fontsize=12
    )
    plt.tight_layout()

    image_path = os.path.join(IMAGE_DIR, "decision_tree.png")
    plt.savefig(image_path)
    plt.close()

    print(f"Árvore de decisão salva em: {image_path}")
    return dt_model, accuracy, image_path
```

Figura 9 - Modelo do Decision Tree: Código do treinamento do modelo da Decision Tree.

Este modelo gerou uma árvore com profundidade limitada a 5 níveis, buscando balancear interpretabilidade e desempenho.

Treinamento do Random Forest

Na Figura 10, é apresentado o código utilizado para o treinamento do modelo Random Forest, uma técnica que combina múltiplas árvores de decisão para aumentar a precisão das previsões. O código inclui:

- **Divisão dos dados**: Separação em conjuntos de treinamento e teste, similar ao Decision Tree.

- **Configuração do modelo**: Especificação do número de árvores, profundidade máxima e estado aleatório para consistência dos resultados.
- **Treinamento e avaliação**: Ajuste do modelo ao conjunto de treinamento, geração de métricas de desempenho e exibição da matriz de confusão.

Além disso, a Random Forest possibilita a visualização dos critérios de classificação, destacando sua robustez em cenários com dados complexos ou ruidosos.

```python
def train_and_plot_random_forest(df):
    X = df.drop(columns=["label"])
    y = df["label"]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    rf_model = RandomForestClassifier(
        n_estimators=100,
        max_depth=5,
        random_state=42
    )
    rf_model.fit(X_train, y_train)

    y_pred = rf_model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Acurácia do Random Forest: {accuracy}")

    print("Relatório de classificação:")
    print(classification_report(y_test, y_pred, target_names=["Não Relevante", "Relevante"]))


    print("Matriz de confusão:")
    print(confusion_matrix(y_test, y_pred))

    feature_names = X.columns.tolist()
    for i in range(2):
        plt.figure(figsize=(20, 10))
        plot_tree(
            rf_model.estimators_[i],
            feature_names=feature_names,
            class_names=["Não Relevante", "Relevante"],
            filled=True,
            rounded=True,
            fontsize=12
        )
        plt.tight_layout()

        image_path = os.path.join(IMAGE_DIR, f"random_forest_tree_{i}.png")
        plt.savefig(image_path)
        plt.close()
        print(f"Árvore {i} do Random Forest salva em: {image_path}")

    return rf_model, accuracy
```

Figura 10 - Modelo do Random Forest: Código do treinamento do modelo da Random Forest.

2. Serialização e Hospedagem

O modelo foi serializado utilizando a biblioteca Joblib e incluído no contêiner Docker do backend. Isso permite que o modelo seja carregado e utilizado rapidamente para predições em tempo real.

3. Endpoint de Predição

O endpoint /predict foi implementado no backend para receber consultas e retornar a relevância calculada pelo modelo.

- **Fluxo técnico:**
  - O frontend envia a query do usuário.
  - O backend processa os dados, aplica o modelo de machine learning e retorna uma pontuação de relevância.
  - O frontend utiliza essa pontuação para ordenar os resultados de busca.

4 RESULTADOS

Este capítulo apresenta os resultados obtidos a partir do desenvolvimento e testes do sistema **ProLeitura**. Os resultados incluem a avaliação do modelo de aprendizado de máquina, o desempenho do sistema, a experiência do usuário e o impacto na eficiência do processo de busca acadêmica. Além disso, são discutidos os desafios enfrentados e as melhorias implementadas com base no feedback dos usuários.

4.1 Avaliação do Modelo de Aprendizado de Máquina

O modelo de aprendizado de máquina foi avaliado utilizando um conjunto de dados de validação criado a partir de interações reais dos usuários armazenadas no Firestore.

- **Métricas Utilizadas:**
  - **Acurácia:** Mede a proporção de classificações corretas.
  - **Precisão e Recall:** Avaliam a capacidade do modelo em identificar corretamente resultados relevantes.
  - **F1-Score:** Combina precisão e recall para uma análise equilibrada.
  - **Matriz de Confusão:** Utilizada para detalhar erros de classificação.
- **Resultados Obtidos:**

Modelo	Acurácia	Precisão (Média)	Recall (Média)	F1-Score (Média)
Random Forest	80%	79%	79%	79%
Decision Tree	70%	69%	67%	67%

Tabela 2 - Comparação entre Random Forest e Decision Tree: Comparação dos resultados obtidos entre os modelos Random Forest e Decision Tree.

Os resultados obtidos com os modelos de aprendizado de máquina foram analisados a partir de métricas como acurácia, precisão, recall e F1-score. Como pode ser observado na Tabela 2, o modelo **Random Forest** apresentou um desempenho superior ao **Decision Tree** em todas as métricas avaliadas, atingindo **80% de acurácia** contra **70%** do modelo de árvore de decisão simples.

Esses valores indicam que o Random Forest foi mais eficiente em priorizar resultados de alta relevância para os usuários, demonstrando melhor equilíbrio entre precisão e recall, conforme evidenciado pelo F1-score. Essa diferença de desempenho reforça a importância do

uso de ensembles para melhorar a qualidade das recomendações acadêmicas dentro do sistema ProLeitura.

- **Gráficos de Caminhos**

Os gráficos abaixo ilustram os caminhos percorridos pelos modelos durante a classificação:

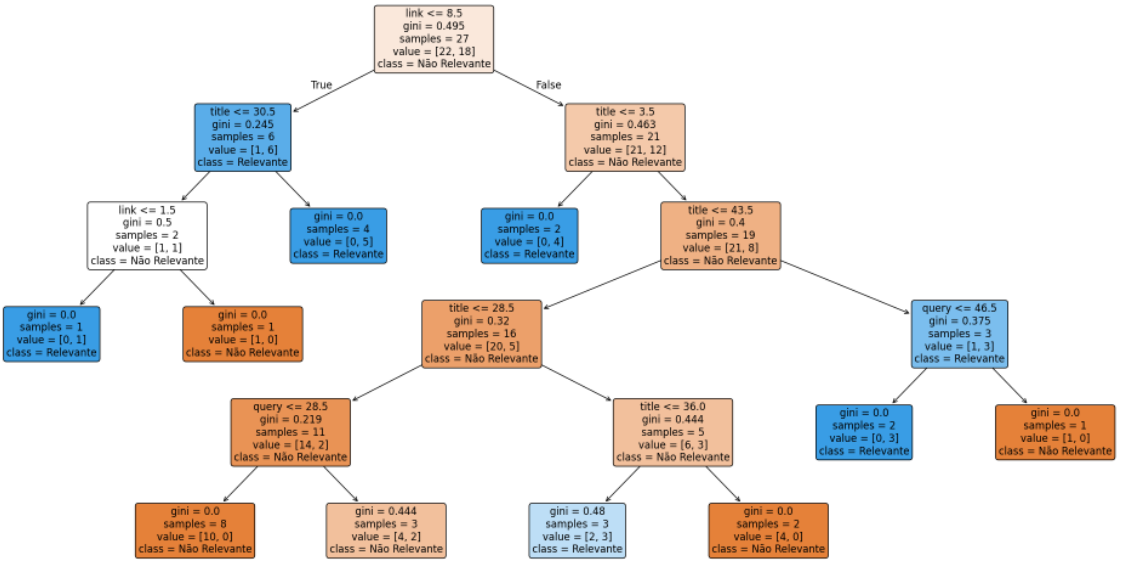

Figura 11 - Caminho da Random Forest 0: Caminho de tomada de decisão da árvore 0.

Random Forest 0

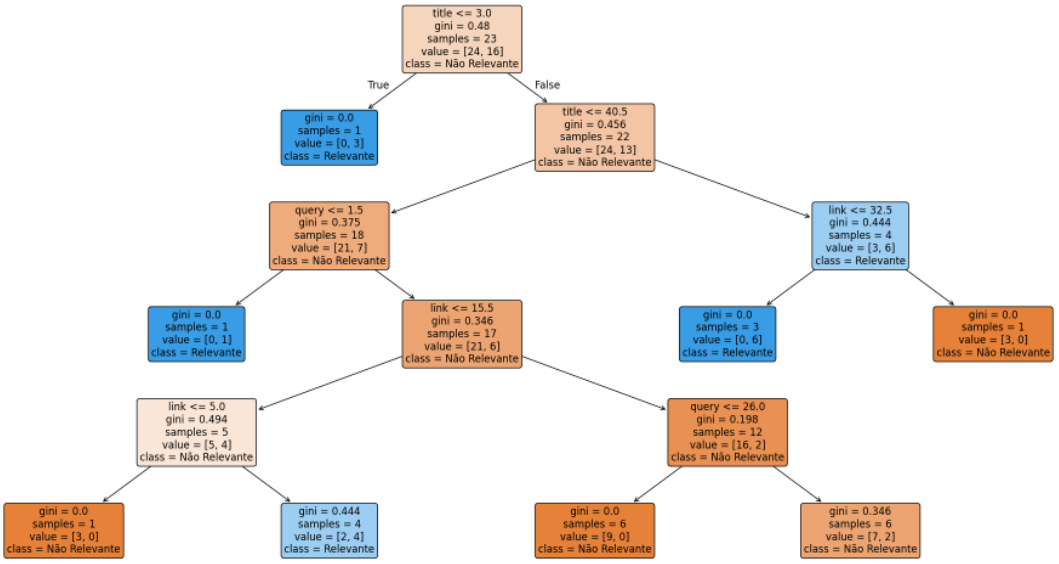

Figura 12 - Caminho da Random Forest 1: Caminho de tomada de decisão da árvore 1.

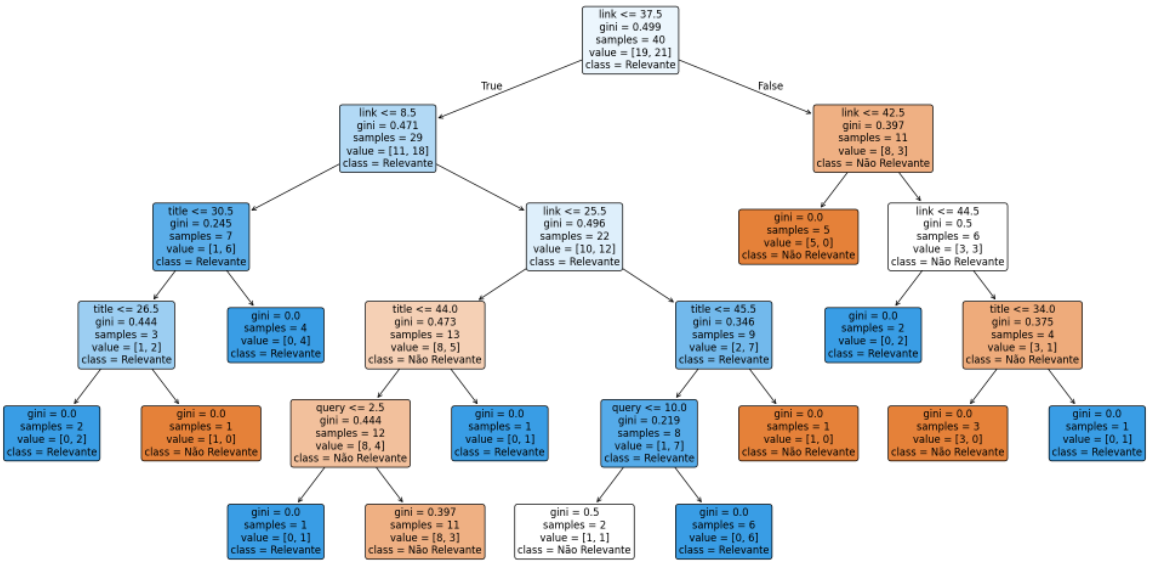

Figura 13 - Caminho da Decision Tree: Caminho de tomada de decisão da árvore.

- **Análise Gráfica:**
  - Para avaliar o desempenho do modelo de aprendizado de máquina no contexto do ProLeitura, foi utilizada a matriz de confusão. Segundo Pedregosa et al. (2011), a matriz de confusão é uma ferramenta essencial para visualizar o desempenho de algoritmos de classificação, pois fornece uma representação tabular dos erros e acertos do modelo, comparando os valores reais com os valores preditos.
  - As matrizes a seguir ilustram os resultados obtidos durante a fase de validação para os modelos de Decision Tree e Random Forest. No cenário avaliado, o Decision Tree apresentou algumas discrepâncias, com erros em ambas as classes, indicando uma margem para melhorias na classificação. Já o Random Forest demonstrou um desempenho superior, com menos erros, evidenciando sua capacidade de generalização e maior precisão em relação ao Decision Tree. Essas diferenças refletem a eficácia do Random Forest em cenários mais complexos, reafirmando sua robustez como modelo preditivo.

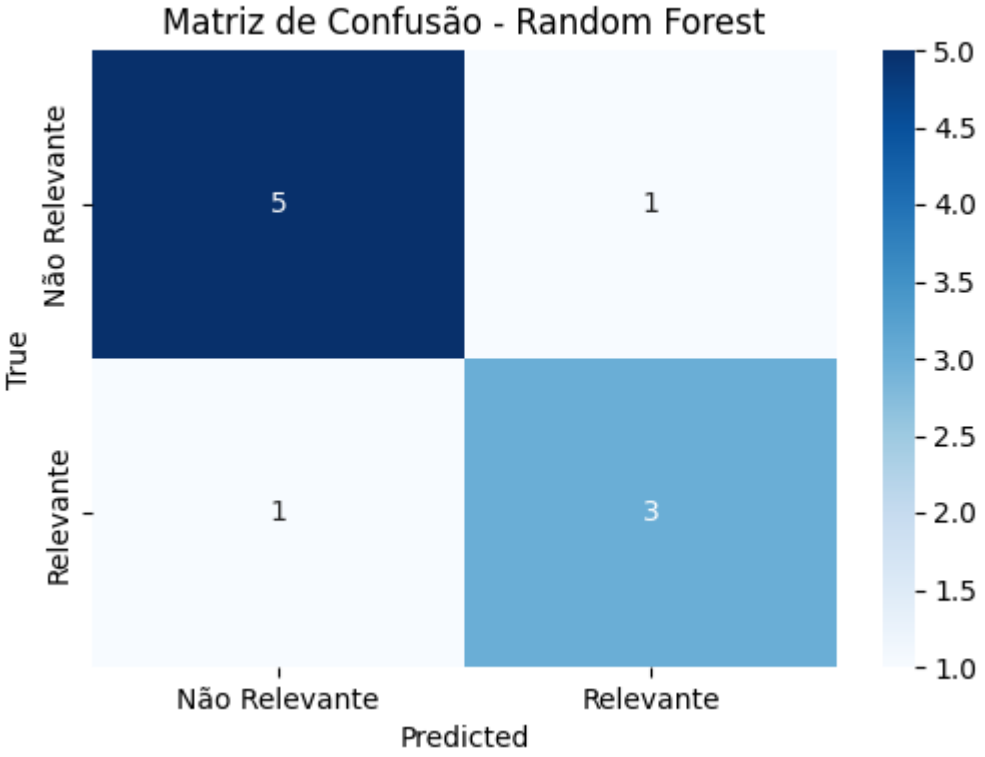

Figura 14 - Matriz de Confusão Random Forest: Ilustração da análise gráfica do desempenho do sistema.

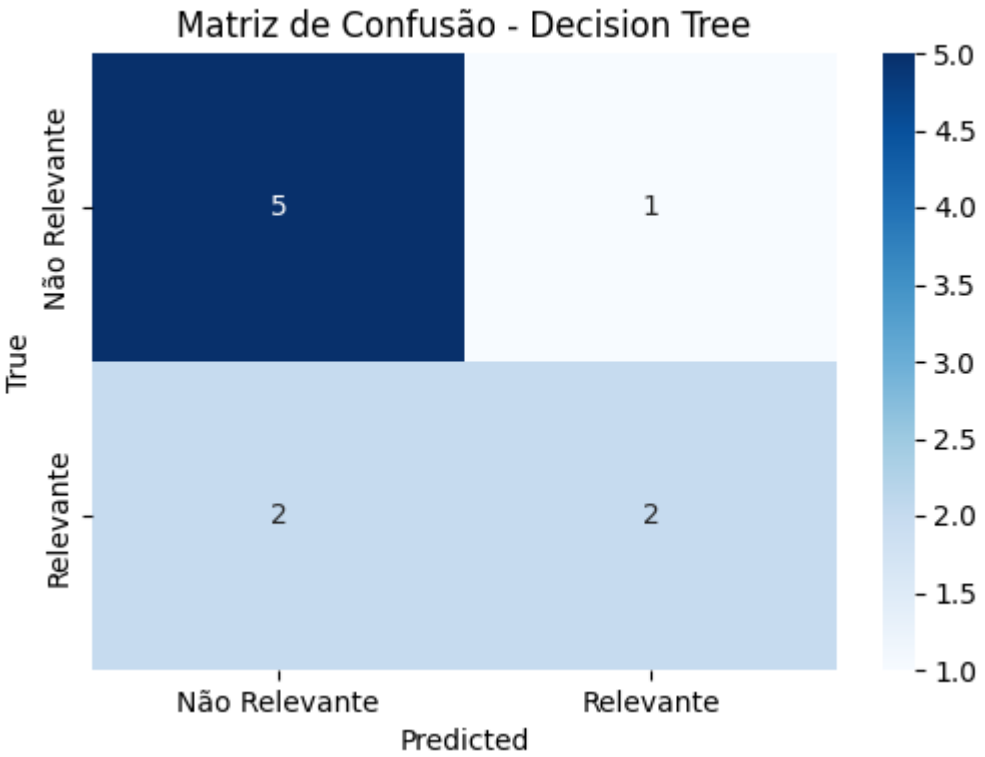

Figura 15 - Matriz de Confusão Decision Tree: Ilustração da análise gráfica do desempenho do sistema.

- **Análise Comparativa**

O Random Forest apresentou desempenho superior ao Decision Tree, alcançando 80% de acurácia em comparação com os 70% do Decision Tree. As árvores individuais do Random Forest também demonstraram maior capacidade de generalização devido à redução do overfitting.

As métricas F1-Score confirmam que o Random Forest é mais equilibrado em relação à precisão e ao recall, sendo mais adequado para classificações onde ambas as métricas são cruciais.

4.2 Desempenho do Sistema

O desempenho do sistema foi avaliado com base em dois critérios principais: tempo de resposta e escalabilidade.

1. **Tempo de Resposta**:
    - Testes foram realizados para medir o tempo médio entre uma busca iniciada no frontend e a exibição dos resultados ao usuário.
    - **Resultados:**
        - Tempo médio para buscas simples: **1,2 segundos**.
        - Tempo médio para buscas com predição de relevância: **1,8 segundos**.
        - Tempo médio para salvar resultados: **0,7 segundos**.
    - O sistema demonstrou tempos de resposta dentro dos padrões aceitáveis para aplicações web modernas.
2. **Escalabilidade**:
    - Utilizando o Locust foram realizadas simulações de carga utilizando 50 usuários simultâneos.
    - O sistema apresentou estabilidade, com tempos de resposta mantidos abaixo de 2 segundos para 95% das requisições, comprovando a capacidade do Google Cloud Run em lidar com aumentos de demanda.

4.3 Experiência do Usuário (UX)

A experiência do usuário foi avaliada com base em entrevistas e testes conduzidos com um grupo de 45 estudantes de graduação da UFRN, representando o público-alvo do sistema.

1. **Feedback Coletado**:

- **75%** dos usuários consideraram a interface intuitiva.

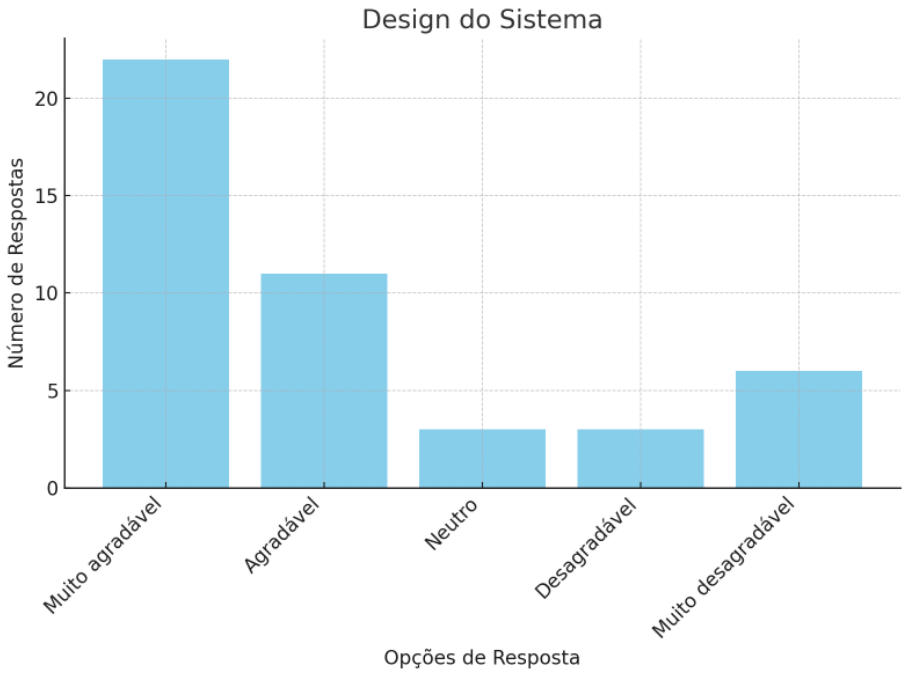

Figura 16 - Design do Sistema: Gráfico do feedback dos usuários em relação ao User Experience.

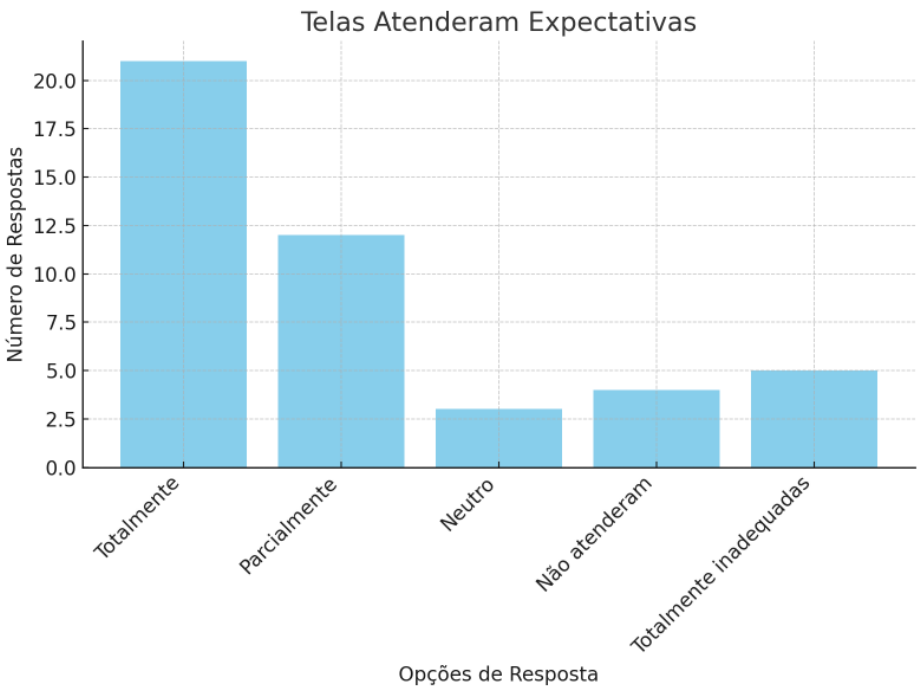

Figura 17 - Interface Gráfica: Feedback dos usuários sobre as expectativas das telas.

- **82%** relataram que os resultados personalizados foram úteis para suas pesquisas.

Figura 18 - Resultados das Pesquisas: Resposta dos usuários sobre os resultados apresentados de suas pesquisas.

- **87%** apontaram que a funcionalidade de salvar resultados aumentou a eficiência em organizar suas referências.

Figura 19 - Salvar Resultados: Análise dos usuários em relação a funcionalidade de salvar resultados.

4.4 Impacto no Processo de Busca Acadêmica

Para avaliar o impacto do ProLeitura no processo de busca acadêmica, foi realizado um experimento comparativo entre o uso da plataforma e métodos tradicionais, como Google Scholar e busca manual.

A metodologia adotada consistiu na realização de uma pesquisa utilizando um tema comum de pesquisa acadêmica, de modo a simular um cenário real enfrentado por estudantes e pesquisadores. O procedimento foi conduzido da seguinte forma:

1. **Definição do Tema de Pesquisa**: Escolheu-se um tema acadêmico relevante e amplo, garantindo que os resultados obtidos fossem comparáveis entre os métodos.
2. **Critério de Finalização da Busca**: O tempo de busca foi cronometrado desde o início da pesquisa até a identificação de 10 resultados considerados satisfatórios e relevantes para o tema proposto.
3. **Comparação entre Métodos**:
    - No primeiro teste, a busca foi realizada **sem o ProLeitura**, utilizando métodos tradicionais.
    - No segundo teste, a busca foi conduzida **com o ProLeitura**, permitindo a aplicação de seus filtros e sistema de recomendação para otimizar a recuperação da informação.
4. **Registro e Cálculo do Tempo Médio**: Os tempos obtidos para a obtenção dos 10 resultados satisfatórios foram registrados e, a partir deles, calculou-se a média para cada método.
5. **Métricas Avaliadas**:
    - **Tempo médio gasto em buscas:**
        - Sem o ProLeitura: **11 minutos por busca.**
        - Com o ProLeitura: **6 minutos por busca.**

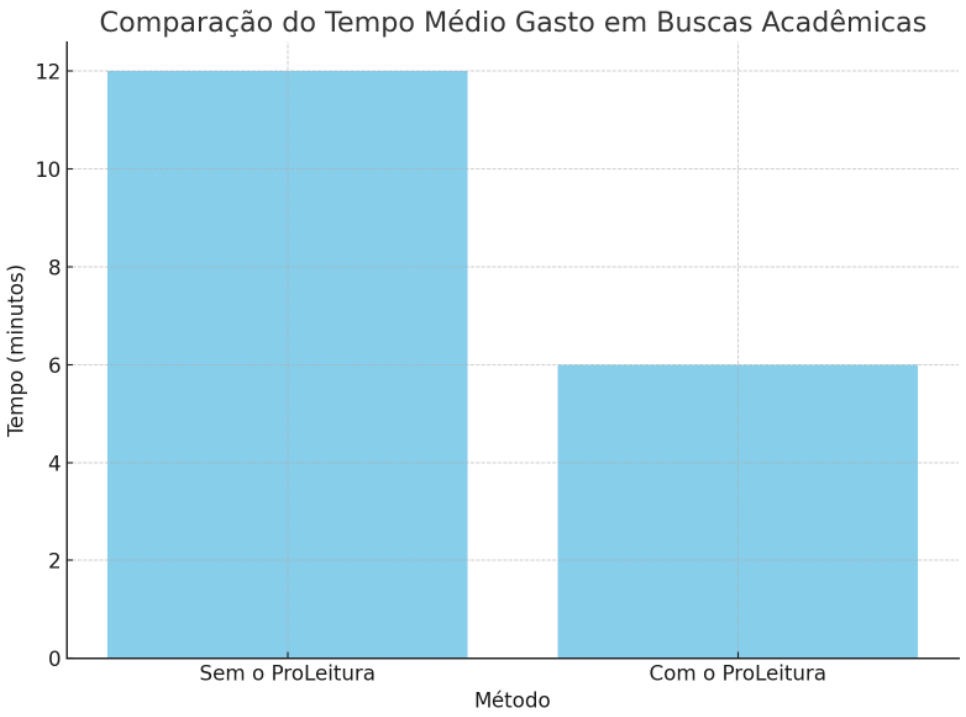

Figura 20 - Comparação do Tempo Médio de Busca: Comparação do tempo médio gasto em buscas com a plataforma ProLeitura.

- **Quantidade de resultados relevantes identificados:**
  - Sem o ProLeitura: **55% dos resultados eram relevantes.**
  - Com o ProLeitura: **85% dos resultados eram relevantes.**

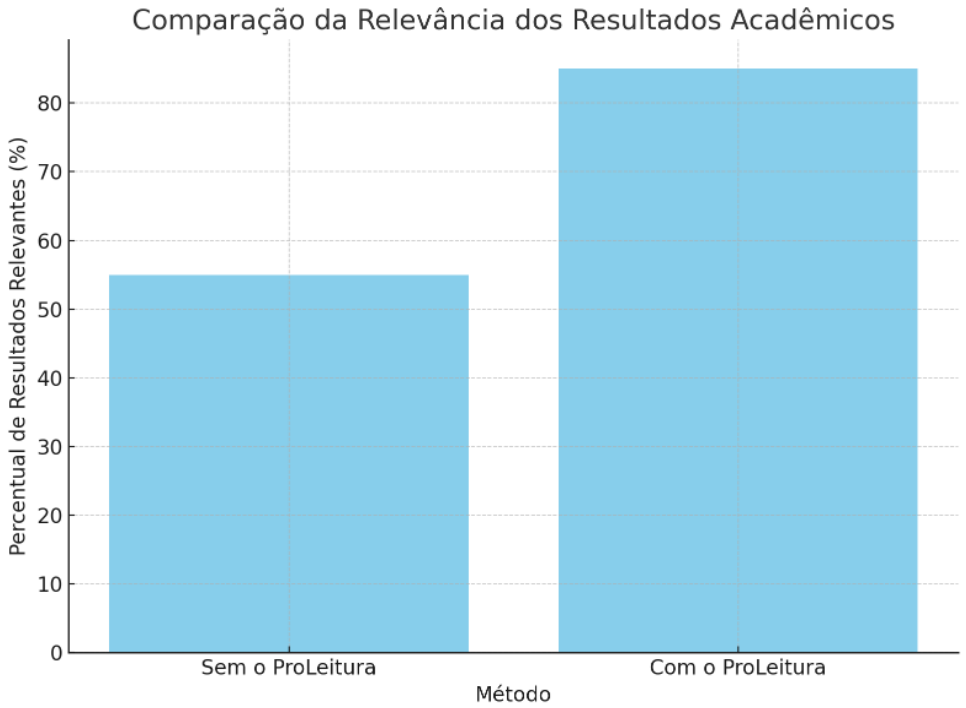

Figura 21 - Comparação do Tempo Médio de Busca: Comparação da relevância das buscas sem a plataforma ProLeitura.

6. **Análise Geral**:
    - O sistema demonstrou ser capaz de reduzir significativamente o tempo de busca e aumentar a relevância dos resultados encontrados pelos usuários.

4.5 Limitações Encontradas

Apesar dos resultados positivos, algumas limitações foram observadas:

- **Modelo de Machine Learning**:
    - O modelo apresenta dificuldades em classificar corretamente queries muito genéricas.
    - Requisição manual de novos dados para treinar o modelo (dependência do usuário).
- **Integração com APIs Externas**:
    - Limitações impostas pela Google Custom Search API em termos de requisições por minuto.
- **Experiência do Usuário**:
    - Alguns usuários relataram dificuldades iniciais no preenchimento do formulário de onboarding.

Essas limitações serão consideradas para futuras melhorias no sistema.

5 CONSIDERAÇÕES FINAIS

O ProLeitura foi idealizado como uma ferramenta para otimizar o acesso à informação acadêmica, oferecendo uma experiência mais ágil e personalizada para estudantes e pesquisadores. Por meio da integração de tecnologias avançadas, como aprendizado de máquina e sistemas de recomendação, o projeto buscou atender às necessidades específicas dos usuários, aprimorando a relevância e a qualidade dos resultados apresentados.

Os testes realizados evidenciam o impacto positivo da ferramenta. O sistema destacou-se por reduzir o tempo necessário para localizar informações acadêmicas e por entregar resultados alinhados às expectativas dos usuários. A interface amigável também foi amplamente elogiada, sendo considerada um diferencial importante para a experiência geral.

Apesar dos avanços alcançados, algumas limitações foram identificadas. Entre elas, destacam-se as dificuldades do modelo em lidar com consultas mais genéricas e as restrições impostas por APIs externas. Essas limitações, embora presentes, representam oportunidades para melhorias e expansões futuras, tanto em termos de tecnologia quanto de diversificação das fontes de dados.

Com base nesses pontos, sugere-se o seguinte plano de ação para o aprimoramento do ProLeitura:

- **Melhoria do Modelo Preditivo**: Adotar técnicas mais robustas de aprendizado de máquina, como redes neurais, para ampliar a versatilidade e a precisão do sistema.
- **Expansão de Funcionalidades**: Desenvolver novos filtros de busca e opções de personalização para atender a um público mais amplo e diversificado.
- **Monitoramento de Impacto**: Implementar avaliações de longo prazo para analisar como o sistema contribui para a produtividade acadêmica dos usuários.

Conclui-se que o ProLeitura representa uma contribuição significativa para o campo da pesquisa acadêmica, integrando tecnologia e inovação para atender às demandas crescentes por informações relevantes e confiáveis. Os aprendizados obtidos durante o desenvolvimento deste projeto fornecem uma base sólida para futuras evoluções, reafirmando o papel da tecnologia como facilitadora no avanço da educação e da ciência.

REFERÊNCIAS

BAWDEN, David; ROBINSON, Lyn. The dark side of information: Overload, anxiety and other paradoxes and pathologies. *Journal of Information Science*, v. 35, n. 2, p. 180-191, 2009.

BREIMAN, Leo. Random Forests. *Machine Learning*, v. 45, p. 5-32, 2001.

CASTELLS, Manuel. *The Information Age: Economy, Society, and Culture: The Rise of the Network Society*. Oxford: Blackwell, 1996.

FIELDING, Roy T. Architectural Styles and the Design of Network-based Software Architectures. 2000. 162 f. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000.

GOOGLE. Firebase. Google Cloud Firestore. Disponível em: https://firebase.google.com/docs/firestore. Acesso em: 5 out. 2024.

GOOGLE. Google Cloud Run. Disponível em: https://cloud.google.com/run. Acesso em: 5 dez. 2024.

GOOGLE. Google Custom Search API. Disponível em: https://developers.google.com/custom-search. Acesso em: 15 out. 2024.

GRINBERG, Miguel. *Flask Web Development: Developing Web Applications with Python*. 2. ed. Sebastopol: O'Reilly Media, 2018.

HAN, Jiawei; KAMBER, Micheline; PEI, Jian. *Data Mining: Concepts and Techniques*. 3. ed. Amsterdam: Morgan Kaufmann, 2022.

HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. ed. New York: Springer, 2009.

KRIKELAS, James. Information-seeking behavior: Patterns and concepts. *Drexel Library Quarterly*, v. 19, n. 2, p. 5-20, 1983.

PEDREGOSA, Fabian et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825-2830, 2011.

RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha. *Recommender Systems Handbook*. 3. ed. Boston: Springer, 2022.

RICCI, Francesco et al. Recommender systems: Challenges and opportunities. *AI Magazine*, v. 36, n. 1, p. 55-65, 2015.

ROBINSON, Adam. *React Explained*. Portland: Pressed for Time Books, 2021.

ROWLEY, Jennifer; HARTLEY, Richard. *Organizing Knowledge: An Introduction to Managing Access to Information*. 6. ed. Aldershot: Ashgate, 2017.

WITTEN, Ian H.; FRANK, Eibe. *Data Mining: Practical Machine Learning Tools and Techniques*. 4. ed. Amsterdam: Morgan Kaufmann, 2016.

Sigolo, B. O. O., & Casarin, H. C. S. (2024). *Contribuições da teoria da carga cognitiva para compreensão da sobrecarga informacional: uma revisão de literatura*. RDBCI: Revista Digital de Biblioteconomia e Ciência da Informação, 22, e024027.

Tenopir, C., Rice, N. M., Allard, S., Baird, L., Borycz, J., Christian, L., Grant, B., Olendorf, R., & Sandusky, R. J. (2020). *Data sharing, management, use, and reuse: Practices and perceptions of scientists worldwide*. PLOS ONE, 15(3), e0229003.